Développement d'Applications Réparties

Chapitre 1 : Les sockets (2)

Séance 3 : 09/10/2026

Sockets TCP : principe

Création d'un Serveur

```
ServerSocket ss = new ServerSocket(2026);
```

```
Socket s = ss.accept();
```

```
InputStream is = s.getInputStream();
int nb =is.read();

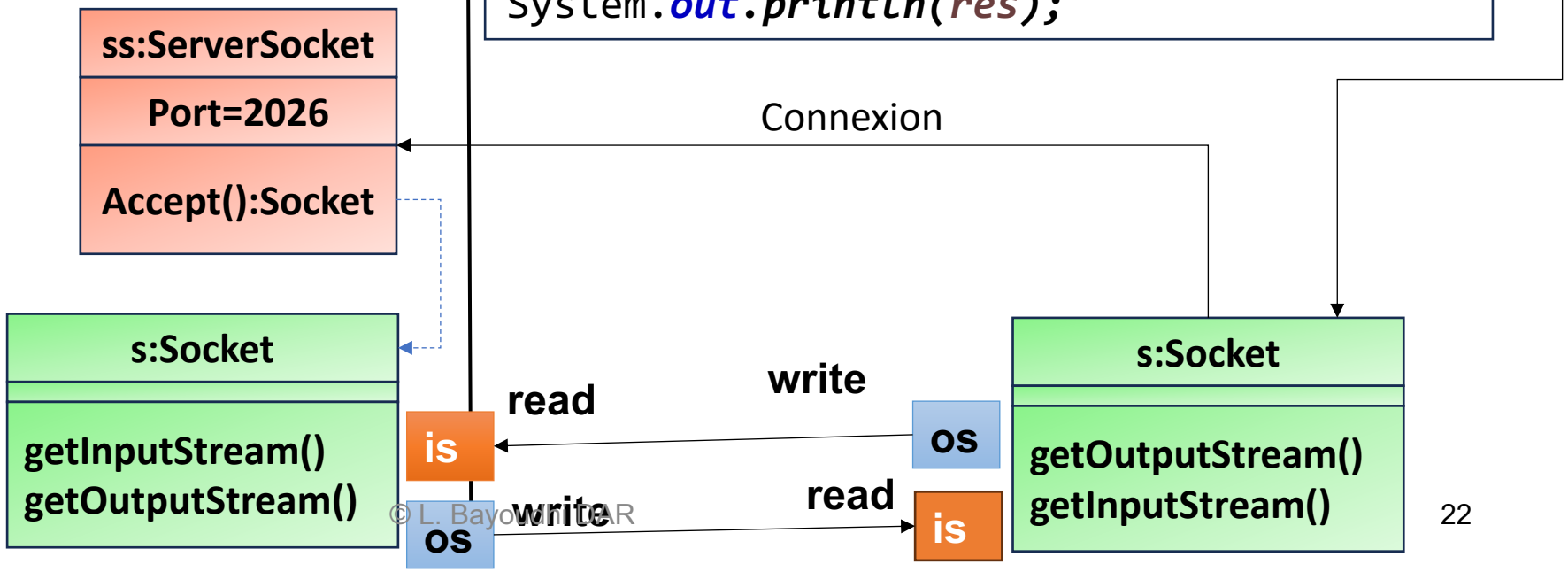
int res = nb*2;
OutputStream os = s.getOutputStream();
os.write(res);
```

Création d'un client

```
Socket s = new Socket("localhost",2026);
```

```
OutputStream os = s.getOutputStream();
os.write(23);

InputStream is = s.getInputStream();
int res = is.read();
System.out.println(res);
```



Ce modèle n'accepte qu'un seul octet ☹

© L. Bayowa NTAR

Sockets TCP : E/R d'une Chaîne de caractères (Flux de CC)

Création d'un Serveur

```
public static void main(String[] args){
// La première étape : création de l'objet ServerSocket
ServerSocket ss = new ServerSocket(2026);
// La deuxième étape : création de l'objet Socket côté serveur suite à la connexion du client
Socket s = ss.accept();
// 3ème étape : lire une CC envoyée par un client
//flux de communication
InputStream is = s.getInputStream();
//flux de traitement
InputStreamReader isr= new InputStreamReader(is);
BufferedReader br = new BufferedReader(isr);
String s= br.readLine();
// La dernière étape : fermeture de la socket
s.close();}
```

Création d'un client

```
public static void main(String[] args){
// La première étape : demande de connexion création de l'objet Socket côté client
Socket s = new Socket("localhost",2026);
// 2ème étape : le client envoie une CC
// flux de communication
OutputStream os = s.getOutputStream();
//flux de traitement
PrintWriter pw = new PrintWriter(os, true);
pw.println( "Bonjour je suis un client");
// La dernière étape : fermeture de la socket
s.close();
```

Sockets TCP : E/R d’objets (flux d’objets)

- La sérialisation est le processus de transformation d'un objet dans un flux d'octets, et la désérialisation est le processus inverse.

- En Java, la sérialisation d’un objet consiste à le convertir en un tableau d’octets qu’on peut écrire sur un fichier ou le transférer sur le réseau (à travers une socket).

➢**Comment peut-on l’implémenter en Java?**

Sockets TCP : E/R d'objets (flux d'objets)

- La classe de l'objet qu'on veut sérialiser doit implémenter l'interface **java.io.Serializable**.
- La classe **ObjectOutputStream** permet de sérialiser un objet.
- **Constructeur :**
  - **ObjectOutputStream(OutputStream os)** permet de créer une instance qui va écrire le résultat de la sérialisation dans le flux fourni en paramètre
- **Méthodes**
  - La méthode **void writeObject(Object obj)** permet de sérialiser un objet. Elle est donc appelée pour écrire un objet sur un flux sérialisé.

Sockets TCP : E/R d'objets (flux d'objets)

- La classe **ObjectInputStream** permet de désérialiser un objet précédemment sérialisé.
- **Constructeur :**
  - **ObjectInputStream(InputStream is)** qui attend en paramètre un objet de type InputStream qui encapsule le flux dans lequel l'objet sérialisé sera lu.
- **Méthodes**
  - La méthode **Object readObject()** permet de désérialiser un objet. Elle est donc appelée pour reconstituer un objet à partir d'un flux sérialisé.

Sockets TCP : E/R d'un objet

```
public class Etudiant implements Serializable{
String nom, prenom;
public Etudiant (String n, String p){
nom = n;
prenom = p; }}
```

Création d'un Serveur

```
public static void main(String[] args){
// La première étape : création de
l'objet ServerSocket
ServerSocket ss = new ServerSocket(2026);
// La deuxième étape : création de l'objet
Socket côté serveur suite à la connexion du
client
Socket s = ss.accept();
// 3ème étape : lire un objet envoyé par un
client
//flux de communication
InputStream is = s.getInputStream();
//flux de traitement
ObjectInputStream ois = new ObjectInputStream(is)
Etudiant v= (Etudiant) ois.readObject();
/*Le serveur doit impérativement connaître la
classe de l'objet à désérialiser*/
// TODO : Traitements et échange
des données
// La dernière étape : fermeture de la socket
s.close();}
```

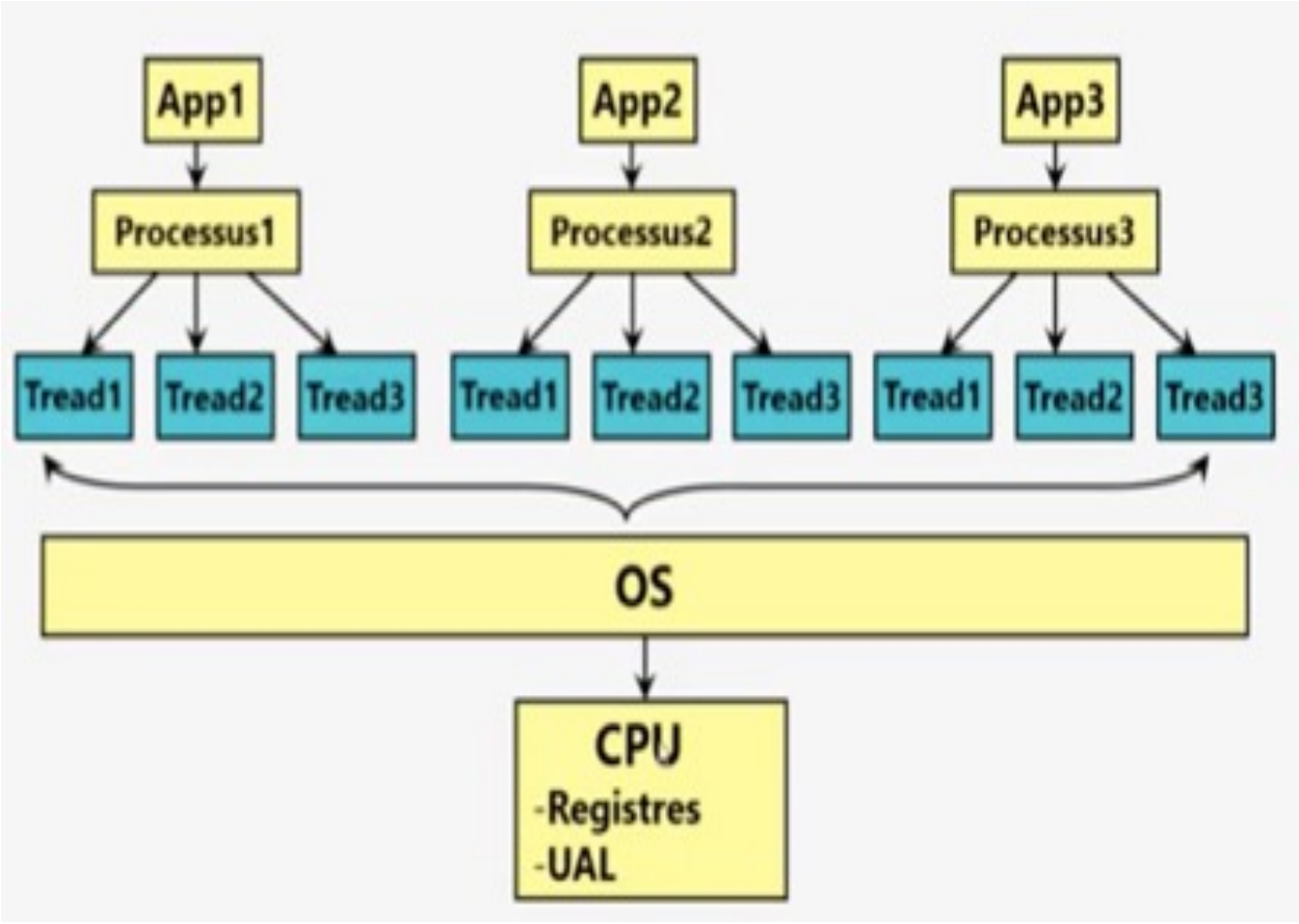
Création d'un client

```
public static void main(String[] args){
// La première étape : demande de connexion
création de l'objet Socket côté client
Socket s = new Socket("localhost",2026);
// 2ème étape : le client envoie un objet Etudiant
// flux de communication
OutputStream os = s.getOutputStream();
//flux de traitement
ObjectOutputStream oos = new ObjectOutputStream(os)
Etudiant e1=new Etudiant ("Ben Mouelli","Hassen");
oos.writeObject(e1);
// La dernière étape : fermeture de la socket
s.close();
```

Serveur multithreads en utilisant les sockets TCP

C’est quoi un thread?

- Pour chaque application démarrée dans un SE, il y a un processus que se démarre
- Dans le cas où on veut exécuter plusieurs tâches en parallèle indépendants, le seul moyen c’est d’utiliser les threads
- Réellement on n’a pas de parallélisme, les threads sont stockés dans une fille d’attente et le SE (OS) gère l’ordonnancement de ces threads, puisque tous les threads ont besoin d’utiliser le même CPU.

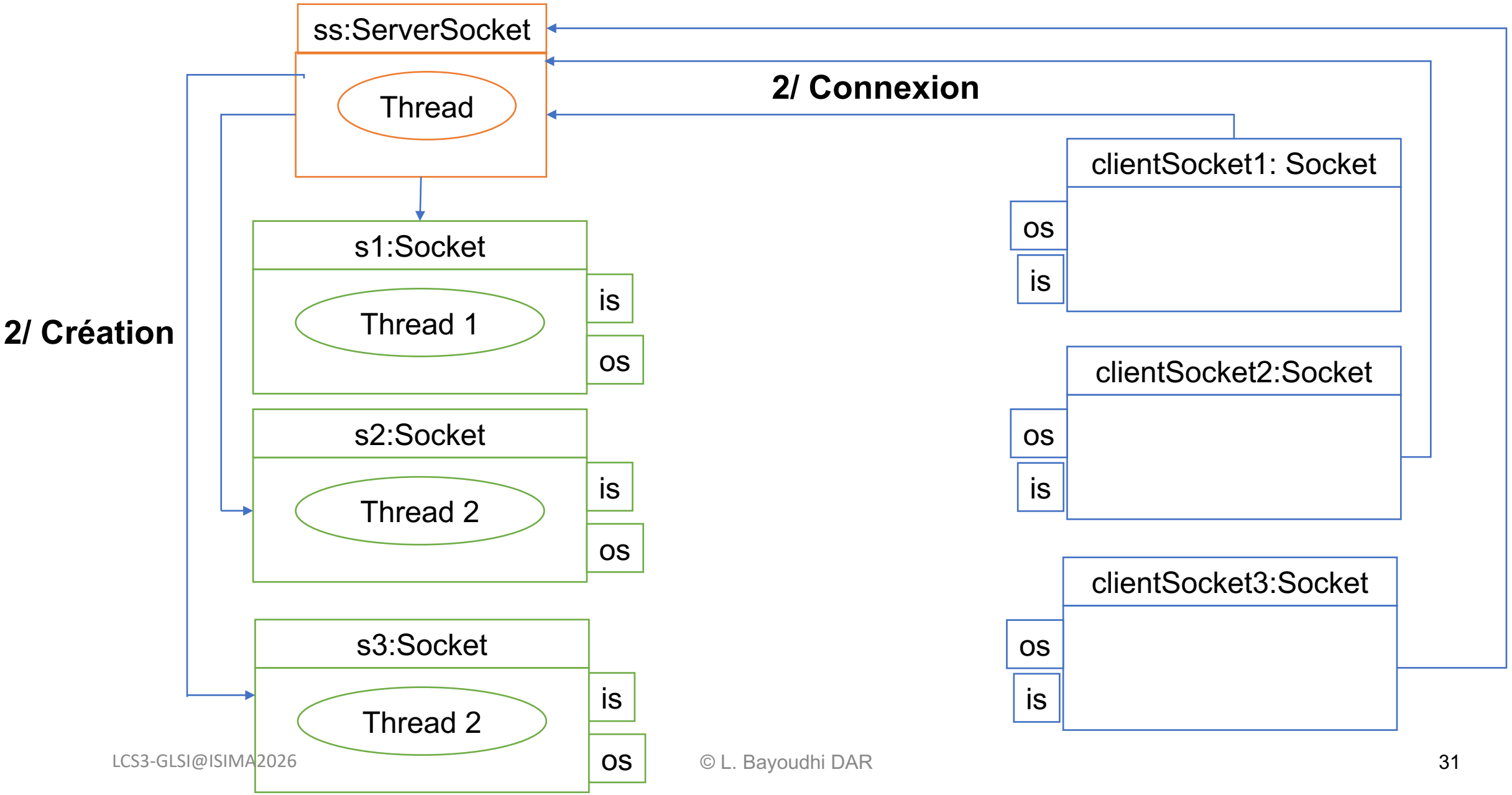


Serveur multithreads en utilisant les sockets TCP

➢ **Comment peut-on l'implémenter en Java?**

- En java, il existe 2 techniques pour créer un thread
  - Soit vous dérivez votre thread de la classe java.lang.Thread (création de threads par héritage)
  - Soit vous implémentez l'interface java.lang.Runnable

Serveur multithreads en utilisant les sockets TCP



© L. Bayoudhi DAR

Serveurs multithreads en utilisant les sockets TCP

- Pour qu'un serveur puisse communiquer avec plusieurs clients en même temps, il faut que :
  - ✓ Le serveur puisse attendre une connexion à tout moment donné.
  - ✓ Pour chaque connexion, il faut créer un nouveau thread associé à la socket du service générée, puis attendre une nouvelle connexion.
  - ✓ Le thread crée doit s'occuper des entrées sorties (read/write) pour se communiquer avec le client indépendamment des autres activités du serveur.

Serveur multithreads - sockets TCP

Création d'un Serveur

```
public class Server extends Thread{
public void run() {

/*La première étape : création de
l'objet ServerSocket*/

ServerSocket ss = new ServerSocket(2026);

 /* La deuxième étape : atttente de
 connexion de plusieurs clients
 Et création de l'objet Socket côté
 serveur suite à la connexion de chaque
 client*/

while (true) {
Socket s = ss.accept();
S.o.p (" un client est connecté" );
new Conversation(s).start();

// La dernière étape : Fermer socket
s.close();
}
```

```
public class Conversation extends Thread{

Socket s;
public Conversation(Socket socket)
{
super();
this.s=socket;}
// Traitement
@Override
public void run() {
S.o.p (" un client est connecté" );
// La dernière étape : Fermer socket
try {
s.close();
} catch (IOException e) {
// TODO Auto-generated catch block
e.printStackTrace();
}} //fin run
 }  //fin class Conversation
public static void main(String[] args){
new Server().start(); //création thread
}
}// Fin de la classe Server
```

Création d'un client

```
public static void main(String[] args){
/*La première étape : création de
 l'objet Socket côté client*/
Socket s = new Socket("localhost",2026);

S.o.p ("je suis un client connecté");

 // échange des données

// La dernière étape : fermer la socket
   s.close();
```


© L. Bayoudhi DAR